Milestone 03 Report: Velocity and Acceleration Kinematics of the Koch v1.1 Arm

German University in Cairo · Faculty of Engineering and Materials Science · Mechatronics Department

MCTR911 Robotics Programming, Winter 2026 · Team 06 · Submitted 11 November 2026

Abstract

The forward and inverse velocity and acceleration kinematics of the 5-joint Koch v1.1 arm were derived, implemented in Python as ROS 2 nodes, and validated against the MuJoCo simulator. All nine validation checks pass: the equations agree with the simulator to better than 1e-5 m, 1e-5 m/s and 1e-4 m/s².

Team member	Student ID
Youssef Hazem	58-2769
Maged Emil	58-3830
Tony Hany	58-6506
Anas Mohamed	58-2897
Ahmed Adham	58-3529

1. Objective and scope

This milestone derives the equations that relate joint motion to end-effector motion at the velocity and acceleration level, in both directions, and tests them on the simulated robot.

- **Forward:** given joint angles, velocities and accelerations, compute the end-effector position, velocity and acceleration.
- **Inverse:** given a desired end-effector position, velocity and acceleration, compute the joint angles, velocities and accelerations.
- **Validation:** feed joint values to MuJoCo, sense the end-effector motion, and compare it with the equations within a tolerance; then do the inverse process.
- **Environment:** continue building the industrial scene around the robot.

The task space is the linear position of the end effector (x, y, z). The joint space is the five arm joints; the gripper is commanded separately. The code is in the ROS 2 package `src/milestone_03` and the validation scripts are in the `Milestone 03` folder.

2. Robot model and kinematic chain

The kinematic chain is taken directly from the robot's MuJoCo model file, so the equations describe exactly the robot being simulated. Each joint is described by two things: where it sits relative to the previous link, and the axis it turns about.

Joint i	Name	Offset from previous link, p_i (m)	Axis, n_i	Range (rad)
1	base_rotation	(0.012, 0, 0.0409)	(0, 0, −1)	−2.2 to 2.2
2	pitch (shoulder)	(0, −0.0209, 0.0154)	(0, 1, 0)	−1.57 to 0.6
3	elbow	(−0.0148, 0.0065, 0.1083)	(0, −1, 0)	−1.57 to 1.45
4	wrist_pitch	(−0.10048, 0.00005, 0.0027)	(0, 1, 0)	−2.0 to 2.0
5	wrist_roll	(−0.045, 0.013097, 0)	(1, 0, 0)	−3.14 to 3.14

The end-effector point is the origin of the wrist_roll frame, the same point used for the forward kinematics in Milestone 02.

The Milestone 02 DH table assumes a zero pose (arm stretched flat along +x) that differs from the simulator's zero pose (upper arm vertical, forearm along −x) and from some of its joint directions. Using the model's own offsets and axes removes the need for angle conversions and lets the equations be compared with the simulator directly.

3. Forward position kinematics

The pose of every joint is built up link by link, starting from the base with $R_0 = I$ and $o_0 = 0$.

```
o_i = o_{i-1} + R_{i-1}\,p_i
```

```
w_i = R_{i-1}\,n_i
```

```
R_i = R_{i-1}\,\mathrm{Rot}(n_i, q_i)
```

Here o_i is the position of joint i in the world frame, w_i is its axis in the world frame, and Rot(n, q) is the rotation by angle q about axis n (Rodrigues' formula). The end-effector position is $p = o_5$ and its orientation is R_5.

At the zero pose this gives p = (−0.1483, −0.0013, 0.1673) m, which matches the simulator.

4. Velocity kinematics

4.1 Geometric Jacobian

If only joint i turns, every point after it moves on a circle about that joint's axis. A point at p then has velocity (axis) × (vector from the axis to the point), and the link's angular velocity is along the axis. Column i of the Jacobian is therefore:

```
J_{v,i} = w_i \times (p - o_i), \qquad J_{\omega,i} = w_i
```

Stacking the five columns gives two 3×5 matrices: J_v for linear velocity and J_ω for angular velocity.

4.2 Forward velocity

The end-effector velocity is the sum of the contributions of all joints:

```
v = J_v(q)\,\dot{q}, \qquad \omega = J_\omega(q)\,\dot{q}
```

Because the end-effector point lies on the wrist_roll axis, the fifth column of J_v is zero: rolling the wrist does not move that point.

4.3 Inverse velocity

J_v has 3 rows and 5 columns, so it cannot be inverted directly: many joint velocities give the same end-effector velocity. The damped pseudo-inverse picks the smallest joint velocity that does the job and stays finite near singular poses:

```
J^{+} = J_v^{T}\left(J_v J_v^{T} + \lambda^{2} I\right)^{-1}, \qquad \dot{q}
= J^{+}\,v
```

The damping factor is λ = 0.001. An optional extra term, $(I - J^{+}J_v)\cdot\dot{q}_0$, moves the joints without moving the end effector and can be used for a secondary goal.

5. Acceleration kinematics

5.1 Forward acceleration

Differentiating v = J_v q̇ with the product rule gives two terms:

$$a = J_v(q)\,\ddot{q} + \dot{J}_v(q,\dot{q})\,\dot{q}, \qquad \alpha = J_\omega\,\ddot{q} + \dot{J}_\omega\,\dot{q}$$

The first term is the acceleration caused by the joint accelerations. The second is present even at constant joint speed; it contains the centripetal and Coriolis effects that arise because the Jacobian itself changes as the arm moves.

5.2 Derivative of the Jacobian

Each column $w_i \times (p - o_i)$ is differentiated with the product rule:

$$\dot{J}_{v,i} = \dot{w}_i \times (p - o_i) + w_i \times (\dot{p} - \dot{o}_i), \qquad \dot{J}_{\omega,i} = \dot{w}_i$$

Every quantity on the right moves only because the joints before it rotate:

$$\Omega_i = \sum_{k<i} w_k\,\dot{q}_k, \qquad \dot{w}_i = \Omega_i \times w_i$$

$$\dot{o}_i = \sum_{k<i} w_k \times (o_i - o_k)\,\dot{q}_k, \qquad \dot{p} = \sum_{k=1}^{5} w_k \times (p - o_k)\,\dot{q}_k$$

Ω_i is the angular velocity of the link that carries joint i, so the axis w_i simply turns with it. The derivative is computed analytically; no numerical differentiation is used in the nodes.

5.3 Inverse acceleration

Solving the forward equation for the joint accelerations with the same damped pseudo-inverse:

$$\ddot{q} = J^{+}\left(a - \dot{J}_v\,\dot{q}\right)$$

6. Inverse position kinematics

The joint angles for a desired end-effector position p_d are found numerically with a damped Newton iteration on the position error:

$$q \leftarrow q + J^{+}\left(p_d - p(q)\right)$$

After each step the angles are clipped to the joint limits. The iteration stops when the error is below 1e-8 m or after 200 steps. Along a trajectory each solve starts from the previous solution, which keeps the joint motion continuous.

7. Worked numerical example

The equations are evaluated at one state to show the size of each term. Angles are in rad, rad/s and rad/s².

Quantity	Joint 1	Joint 2	Joint 3	Joint 4	Joint 5
q	0.3	−0.4	0.5	0.2	0.1
$\dot{q}$	0.2	−0.1	0.3	0.1	0
$\ddot{q}$	0.1	0.2	−0.1	0	0.1

Linear Jacobian J_v at this pose (m/rad)

	Joint 1	Joint 2	Joint 3	Joint 4	Joint 5
x	0.0445	−0.0115	0.1013	−0.0277	0
y	0.1483	0.0036	−0.0313	0.0086	0
z	0	0.1548	−0.0990	0.0344	0

Results

Quantity	x	y	z
Position p (m)	−0.1363	0.0445	0.0443
Velocity v = J_v $\dot{q}$ (m/s)	0.0377	0.0208	−0.0417
Angular velocity ω (rad/s)	−0.0887	−0.2866	−0.2000
J_v $\ddot{q}$ (m/s²)	−0.0080	0.0187	0.0409
J̇_v $\dot{q}$ (m/s²)	0.0157	−0.0174	0.0140
Acceleration a (m/s²)	0.0078	0.0012	0.0549

The J̇_v $\dot{q}$ term is as large as the J_v $\ddot{q}$ term here, so leaving it out would give a wrong acceleration.

Inverse check. Feeding v back through the pseudo-inverse gives $\dot{q}$ = (0.2000, −0.0912, 0.2538, −0.0726, 0). This differs from the original joint velocities because the arm is redundant for a 3D position task, but it reproduces the same v exactly; it is the smallest joint velocity that does so.

8. ROS 2 implementation

The equations live in one module, `kinematics.py`, and five nodes connect them to the simulator. All topics are under `low_cost_robot_arm/`.

Node	Role	Subscribes to	Publishes
`sim_node`	MuJoCo simulator with the environment and viewer	`joint_commands`, `joint_kinematics_cmd`	`joint_state`, `joint_kinematics`, `ee/sim`
`velocity_kinematics_node`	Forward equations: FK, J $\dot{q}$, J $\ddot{q}$ + J′ $\dot{q}$	`joint_kinematics`	`ee/equations`
`trajectory_node`	Desired end-effector position, velocity and acceleration (circle or pick-and-place)		`ee/desired`, `trajectory_done`
`inverse_kinematics_node`	Inverse position, velocity and acceleration	`ee/desired`	`joint_commands`, `joint_kinematics_cmd`
`validation_node`	Compares the three end-effector streams and writes PASS/FAIL, plots and a summary	`ee/sim`, `ee/equations`, `ee/desired`, `trajectory_done`	

Data flow. The trajectory node publishes the desired motion. The inverse node turns it into joint values. The simulator applies them and publishes both its joint state and what it senses at the end effector. The forward node recomputes the end-effector motion from the joint state with the equations. The validation node compares sensed, computed and desired values sample by sample, paired by a shared time stamp.

Two simulator modes

- **Kinematic mode:** joint positions, velocities and accelerations are imposed directly, with no physics. This isolates the kinematic equations and is used for the pass/fail checks.

- **Dynamic mode:** position servos and full physics. The forward check still applies; the inverse comparison is reported as a tracking error, because the servos lag behind the command.

How to run

```
colcon build --packages-select milestone_03 && source install/setup.bash
ros2 launch milestone_03 milestone_03.launch.py                        #
kinematic validation, circle
ros2 launch milestone_03 milestone_03.launch.py mode:=dynamic          #
physics and servo tracking
ros2 launch milestone_03 milestone_03.launch.py trajectory:=pick_place
mode:=dynamic
python3 "Milestone 03/validate_simulation.py"                          #
offline validation, no ROS
python3 "Milestone 03/demo_viewer.py"                                  #
pick-and-place demo in the viewer
```

9. Validation against the simulator

All nine checks pass. The forward equations agree with MuJoCo to numerical precision, and the inverse equations reproduce a desired Cartesian circle within the tolerances.

Tolerances: 1e-5 m for position, 1e-5 m/s for velocity, 1e-4 m/s² for acceleration.

What the simulator senses. A marker (site) is placed at the end-effector point. MuJoCo reports its position, its velocity and its acceleration. MuJoCo's acceleration output behaves like an accelerometer (it excludes gravity), so gravity is added back to obtain the true kinematic acceleration.

9.1 Tests

1. **Forward, random states (tests 1 to 3).** 300 random states are imposed on the simulator: joint angles within 70% of their ranges, joint velocities up to ±2 rad/s, joint accelerations up to ±5 rad/s². The sensed end-effector position, velocity and acceleration are compared with FK(q), J_v q̇ and J_v q̈ + J̇_v q̇.
2. **Inverse, Cartesian circle (tests 4a to 4c).** A circle of radius 0.03 m centred at (−0.15, 0, 0.12) m in the y-z plane, period 2 s, is sampled at 201 points. Each sample is converted to joint angles, velocities and accelerations with the inverse equations. These are imposed on the simulator and the sensed end-effector motion is compared with the desired circle.

3. **Dynamic run (tests 5a to 5c).** The arm is simulated for 8 s with full physics and position servos following sinusoidal joint references. The equations are evaluated on the simulator's own joint angles, velocities and accelerations and compared with the sensed end-effector motion.

9.2 Results

Test	Maximum error	Mean error	Tolerance	Result
1. Forward position	1.1e-16 m	3.8e-17 m	1e-5 m	Pass
2. Forward velocity	3.4e-16 m/s	7.7e-17 m/s	1e-5 m/s	Pass
3. Forward acceleration	5.4e-15 m/s²	1.4e-15 m/s²	1e-4 m/s²	Pass
4a. Inverse position	5.2e-10 m	5.0e-10 m	1e-5 m	Pass
4b. Inverse velocity	3.6e-6 m/s	3.0e-6 m/s	1e-5 m/s	Pass
4c. Inverse acceleration	1.1e-5 m/s²	9.5e-6 m/s²	1e-4 m/s²	Pass
5a. Dynamic position	9.0e-17 m	3.5e-17 m	1e-5 m	Pass
5b. Dynamic velocity	1.1e-15 m/s	5.2e-17 m/s	1e-5 m/s	Pass
5c. Dynamic acceleration	5.4e-14 m/s²	2.4e-15 m/s²	1e-4 m/s²	Pass

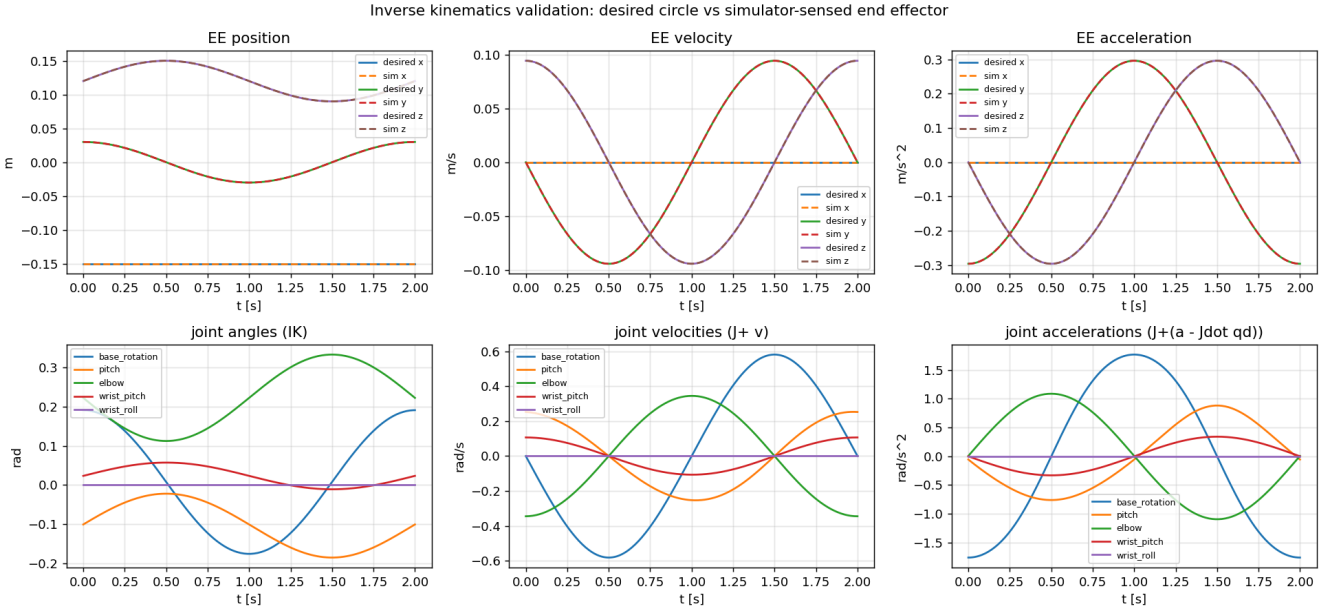


Figure 1. Inverse validation (test 4). Top row: desired (solid) and simulator-sensed (dashed) end-effector position, velocity and acceleration lie on top of each other. Bottom row: joint angles, velocities and accelerations produced by the inverse equations.

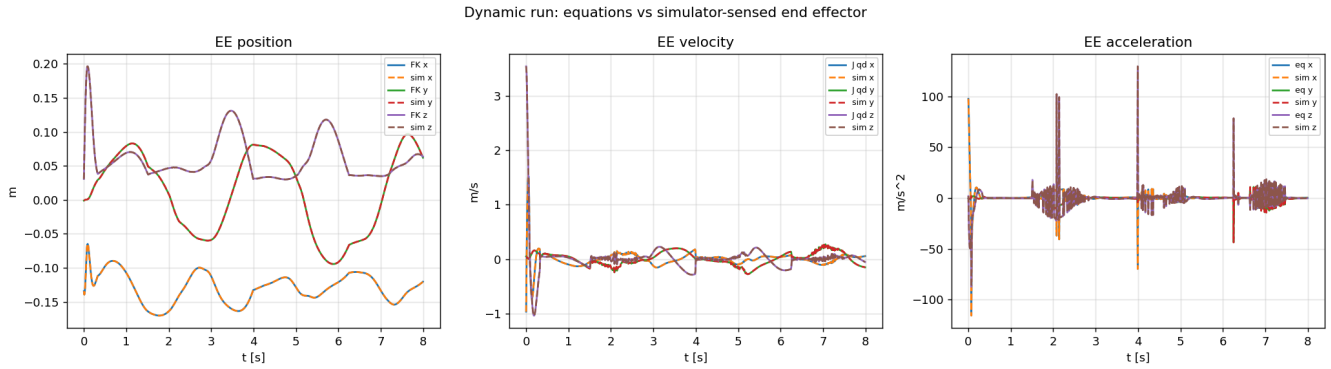


Figure 2. Dynamic run (test 5). Equations (solid) and simulator-sensed values (dashed) coincide for position, velocity and acceleration, including through the contact spikes.

9.3 Comments on the results

- **Forward errors are at machine precision.** The equations and the simulator use the same geometry, so the only difference is floating-point rounding.
- **Inverse errors are small but not zero.** They come from the damping in the pseudo-inverse ($\lambda = 0.001$), which trades a little accuracy for stability near singular poses. The inverse position error is set by the iteration's stopping threshold.
- **The dynamic run shows large spikes** in velocity (up to about 3.7 m/s) and acceleration (about ±100 m/s²), at the start and near 2, 4 and 6 s. In this test the arm starts with the gripper touching the floor and is in contact with it for about 30% of the run, which produces the impacts. The equations still match the simulator through the spikes, because they are evaluated on the simulator's actual joint state. See section 11.
- **Pick-and-place demo.** With closed-loop inverse velocity control, $\dot{q} = J^{+}(v_d + K_p(p_d - p))$ with $K_p = 20\ s^{-1}$, the end effector follows the conveyor-to-bin path with a maximum tracking error of 16.5 mm. This error is servo lag in the physics simulation, not a kinematics error.

10. Simulation environment

The industrial scene is a desktop pick-and-place cell sized to the arm's reach of about 0.25 m. It is built in Python (`sim_model.py`) on top of the robot model and can be switched on or off from the launch file.

Component	Description
Conveyor	Belt in front of the arm with side rails, legs and end rollers
Workpieces	Four coloured blocks riding the belt at 0.02 m/s and wrapping around

Component	Description
Bin	Open box beside the arm where parts are dropped
Tool rack	Back panel with four tools behind the arm
Operator	Worker figure with a helmet standing at the end of the belt
Chair	Seat, backrest and four legs

Application motion. The pick-and-place trajectory moves the end effector above the belt, down to pick, up, across to the bin, down to place, and back. Each segment uses a minimum-jerk profile, so velocity and acceleration are continuous and zero at every stop. The circle trajectory is used for the kinematic validation.

11. Limitations

- **Position only.** The task space is the end effector′s linear position. The angular Jacobian is derived and computed, but orientation is not controlled or validated.
- **Inverse position depends on the starting guess.** Along a trajectory each solve starts from the previous solution and converges. From a fixed default guess, 113 of 2,000 random reachable targets (5.7%) did not converge to within 0.1 mm.
- **Floor contact in the dynamic test.** The sinusoidal references of test 5 start with the gripper touching the floor, which causes the spikes noted in section 9.3. The kinematic comparison is unaffected, but the references should be raised so the arm moves freely.
- **Visual-only environment.** The environment objects have no collision, and the workpieces are moved by the belt script. The gripper opening and closing is illustrative; parts are not physically grasped yet.
- **No DH parameters in this milestone.** The chain is taken from the model file, not from the Milestone 02 DH table, for the reason given in section 2.
- **Singularities.** They are handled only through the damping in the pseudo-inverse; closeness to a singularity is not monitored along the trajectory.

12. Conclusion

The forward and inverse velocity and acceleration kinematics of the Koch v1.1 arm were derived from its geometric Jacobian and the Jacobian′s analytical time derivative, implemented as ROS 2 nodes, and confirmed against MuJoCo. Forward results match the

simulator to machine precision and inverse results reproduce a Cartesian circle within the stated tolerances. The pick-and-place cell and its trajectory are in place for Milestone 04, where trajectory tracking and control will be added.